SmartReply Agent — Document de conception

Version 1.0 · 6 novembre 2026 · Koffa Jean Agoudavi

Portée	État actuel vérifié + cible post-MVP explicitement non implémentée.
Référence	Commit 02e779c · tête Alembic a9c4e7f2b6d1.
Diffusion	Document partageable ; aucun secret ni identifiant externe.

1. Vue d’ensemble

SmartReply Agent est conçu comme un SaaS multi-utilisateur qui prépare le traitement des emails professionnels sans retirer le contrôle humain. Le système reçoit des messages Gmail, élimine les messages non pertinents, produit une analyse structurée avec Gemini, conserve les données métier dans le Google Sheet propre à l’entreprise et crée un brouillon Gmail. Telegram peut notifier et porter des actions explicites lorsque l’utilisateur l’a connecté.

Ce document présente deux niveaux volontairement distincts : l’architecture actuelle, vérifiée dans le dépôt au commit de référence, puis une cible post-MVP issue des fonctions reportées du cadrage. Les composants cibles sont des propositions de conception à valider ; ils ne décrivent ni du code existant ni un état de production vérifié.

Principe	Conséquence de conception
Tenant dérivé	L’entreprise vient du JWT ou d’une relation interne vérifiée, jamais d’un identifiant libre fourni par le client.
Contrôle / métier séparés	Neon porte identité, règles et idempotence ; Gmail et le Sheet portent le contenu métier.
Effets externes marqués	Chaque effet non transactionnel est précédé ou suivi d’un marqueur Neon permettant une reprise bornée.
Humain dans la boucle	Le pipeline crée un brouillon ; aucun envoi automatique n’est déclenché par le traitement.
État actuel ≠ cible	Les schémas verts/pleins sont implémentés ; les composants ambrés ou en pointillés de la cible restent à valider.

Portée vérifiable

Baseline documentaire : commit 02e779cd57a1f2720aac0b85be3201ebc0d457f5 · tête Alembic a9c4e7f2b6d1. Les consoles Render, Neon, Google, Telegram et Redis de production n’ont pas été auditées pour ce document.

2. Architecture actuelle

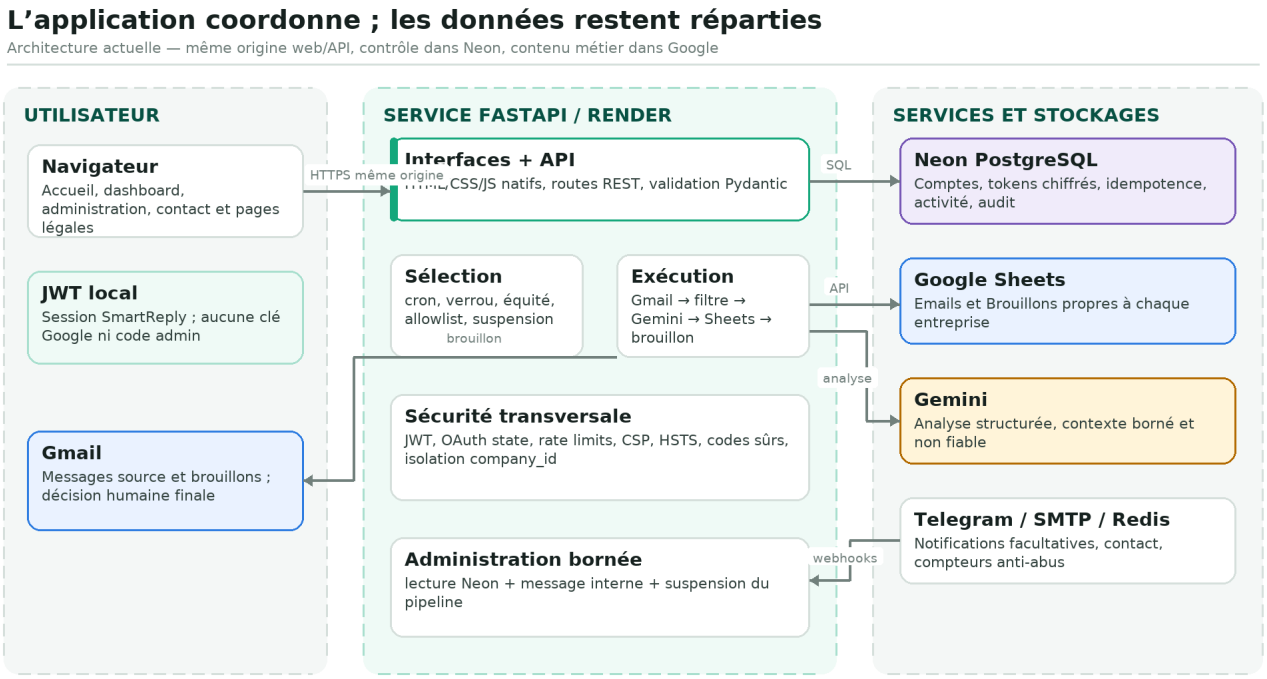


Schéma 1 — Architecture logique actuelle et responsabilités des quatre plans : client, application, contrôle et services externes.

FastAPI sert les interfaces et l’API sur une même origine. Cette simplicité réduit les frontières CORS et permet une politique CSP connect-src limitée à self. Le service coordonne plusieurs stockages : il ne transforme ni Neon en boîte mail, ni Google Sheets en base d’authentification.

Zone	Responsabilité	Comportement si indisponible
Navigateur	Interaction, JWT SmartReply, affichage sûr.	Les traitements serveur continuent ; les pages déjà chargées perdent les lectures API.
FastAPI / Render	API, orchestration, contrôles et pages statiques.	Aucun nouveau passage ; les données Gmail/Sheets existantes restent chez leurs fournisseurs.
Neon PostgreSQL	Identité, configuration, idempotence et audit.	/ready échoue et le pipeline ne doit pas progresser.
Google / Gemini	Sources, stockage métier, brouillons et analyse.	Échec borné par entreprise ou par message ; reprise selon le type d’erreur.
Telegram / SMTP / Redis	Canaux facultatifs et compteurs.	La panne Telegram ou Redis n’empêche pas le traitement Gmail/Sheet.

3. Diagramme de cas d’utilisation

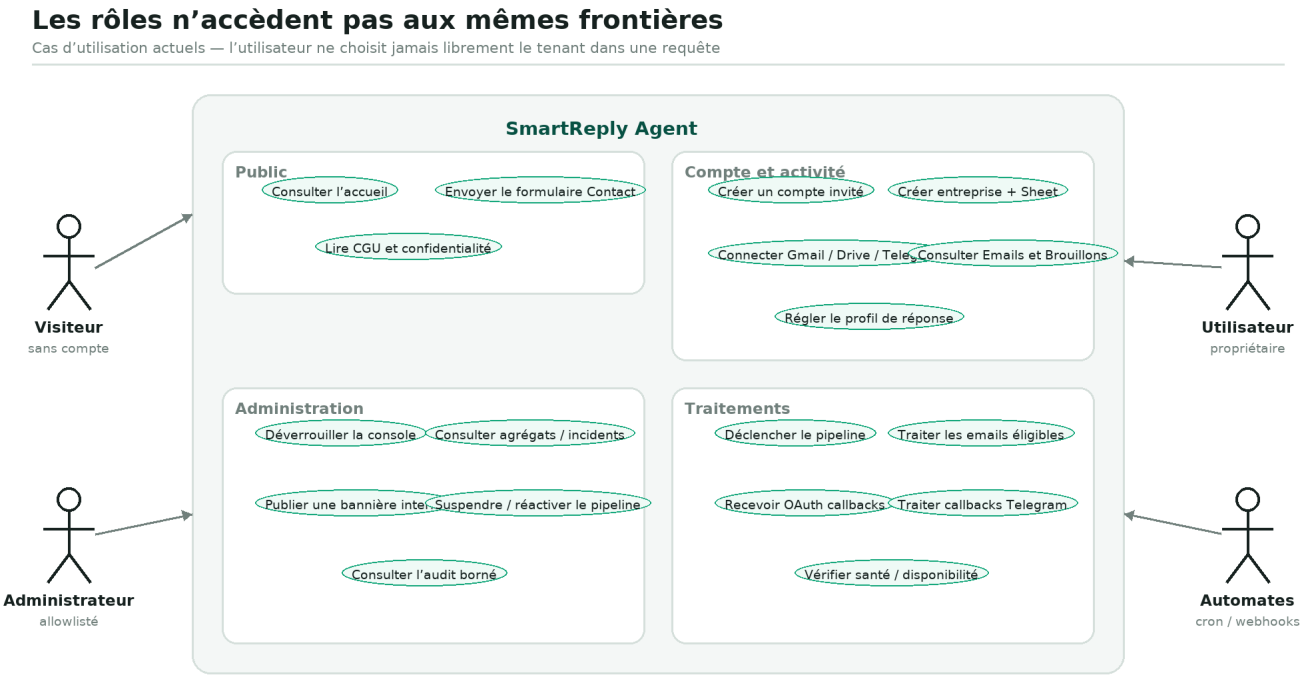


Schéma 2 — Acteurs et cas principaux. Les automates utilisent des secrets dédiés ; ils ne possèdent pas de session utilisateur.

Acteur	Frontière d’accès	Cas notables
Visiteur	Routes publiques.	Accueil, demande de contact, CGU et confidentialité.
Utilisateur	JWT SmartReply ; tenant dérivé.	Onboarding, intégrations, dashboard, Emails, Brouillons et Paramètres.
Administrateur	JWT + allowlist + code secondaire.	Agrégats, incidents, message interne et suspension du pipeline.
Cron / Google / Telegram	Secrets serveur, state OAuth ou secret webhook.	Déclenchement du pipeline, callbacks OAuth et callbacks Telegram.

Frontières d’interface

Les pages Contact, CGU et confidentialité restent publiques. La console /admin/ n’est liée ni depuis l’accueil ni depuis la sidebar utilisateur.

4. Modèle de données actuel

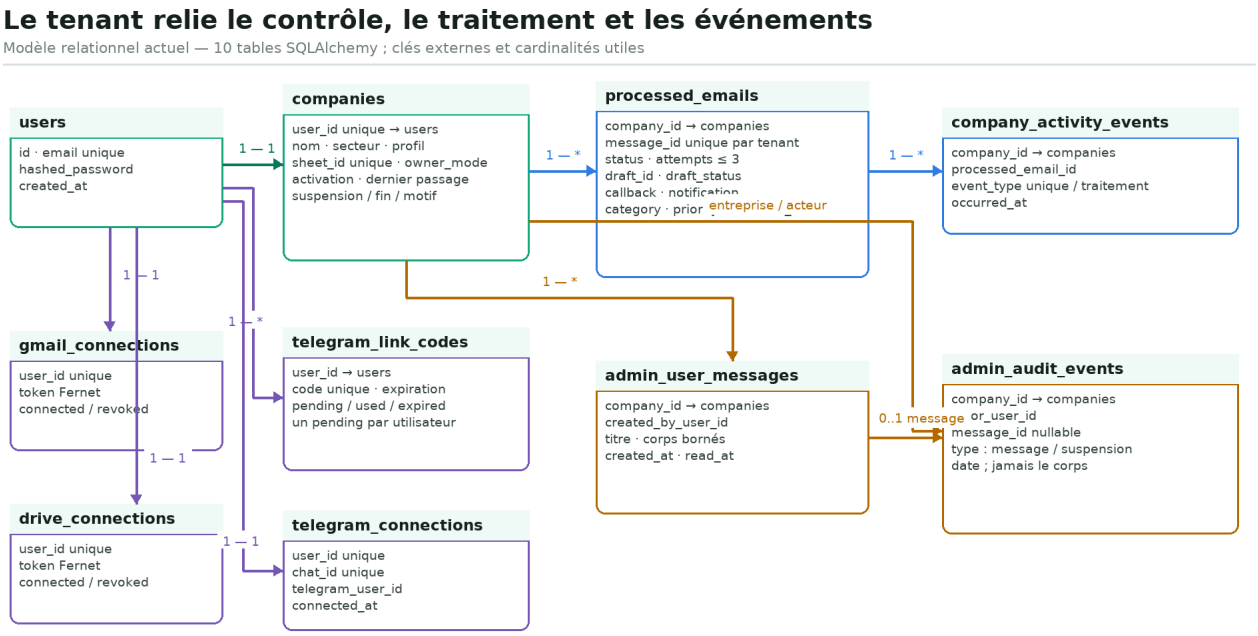


Schéma 3 — Modèle relationnel simplifié des dix tables SQLAlchemy. Les champs de contenu Gmail et Brouillons ne sont pas recopiés dans Neon.

Le modèle organise les données autour de users et companies. Un utilisateur possède au plus une entreprise, une connexion Gmail, une connexion Drive et une liaison Telegram. processed_emails est le registre central de dédoublonnage et company_activity_events le modèle de lecture analytique minimal.

Contrainte	Effet
users.email unique	Une identité SmartReply par adresse.
companies.user_id unique	Une entreprise par utilisateur dans le MVP.
processed_emails(company_id, gmail_message_id) unique	Le même message ne peut être réservé deux fois pour un même tenant.
callback_token unique et opaque	Aucun ID Gmail, Sheet ou entreprise n’est encodé dans un bouton Telegram.
company_activity_events(traitement, type) unique	Une transition métier réussie produit au plus un événement analytique.
Suppression en cascade depuis companies	Registres, événements et messages associés restent cohérents.

4.1 Cartographie des stockages et classification

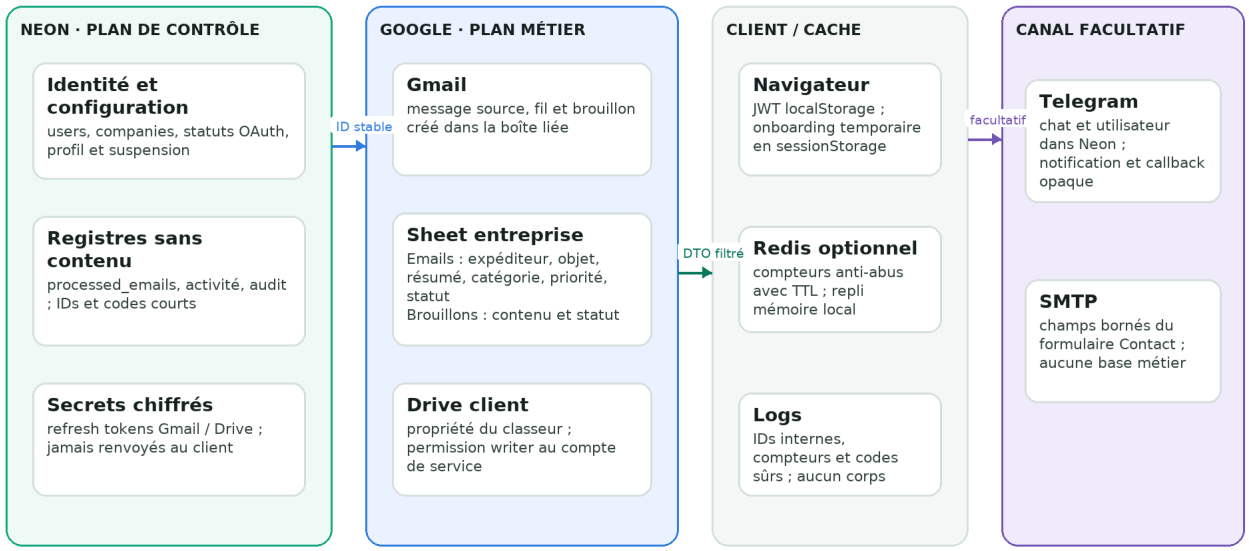


Schéma 4 — Localisation des données et règle de minimisation appliquée à chaque stockage.

Donnée	Source de référence	N’est pas exposé à
Mot de passe	Hash Argon2id dans Neon.	Administrateur, logs et services Google.
Refresh tokens	Valeur Fernet dans Neon.	Navigateur, API publique, administration et Sheets.
Corps email	Gmail ; transmis borné à Gemini pendant le traitement.	Neon, réponses admin et journaux.
Résumé / brouillon	Sheet de l’entreprise et brouillon Gmail.	Administration et autres tenants.
Analytics	processed_emails + activity_events.	Contenu d’email, adresse complète et identifiants externes.
Message admin	admin_user_messages pour la bannière.	Audit : celui-ci ne garde que type, acteur et date.

5. Onboarding et intégrations

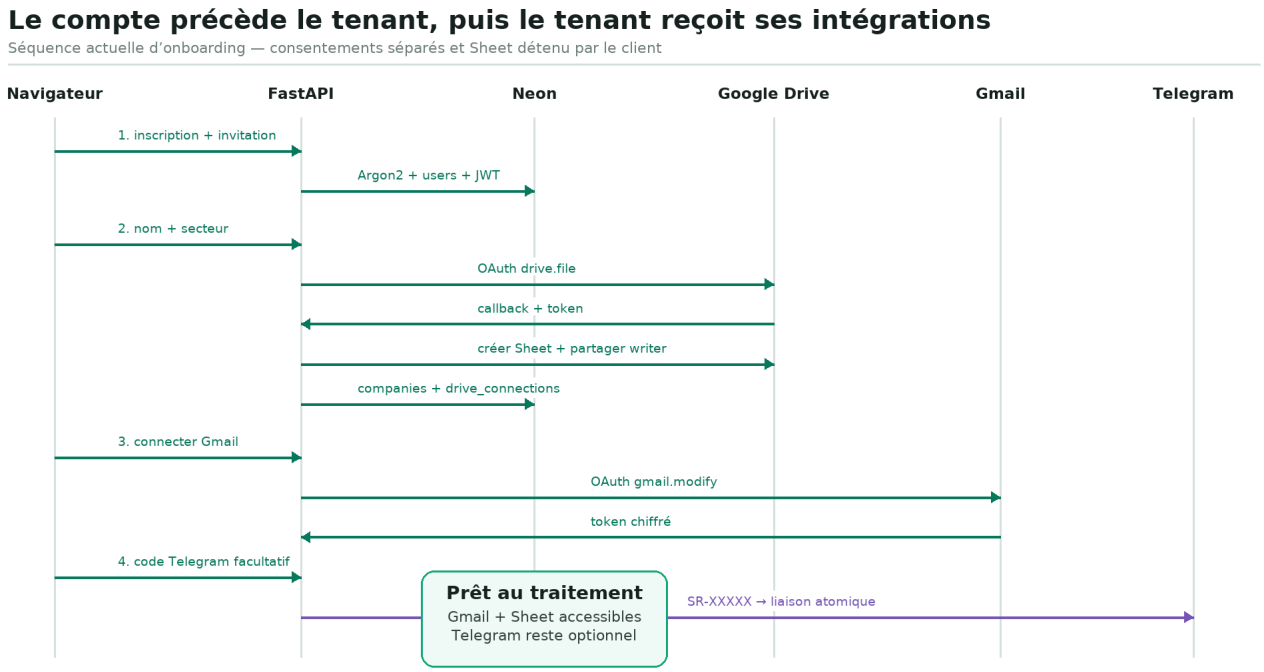


Schéma 5 — Séquence d'onboarding. Drive et Gmail ont des consentements séparés ; Telegram est facultatif.

Le navigateur conserve temporairement le nom et le secteur pendant l'aller-retour Drive. Le callback stocke le refresh token chiffré côté serveur. La création de l'entreprise génère le classeur dans le Drive autorisé du client, ajoute la permission writer du compte de service et écrit companies seulement après réussite de l'ensemble.

Intégration	Prérequise pour le pipeline	Donnée de contrôle
Gmail	Oui	gmail_connections.status = connected et refresh token chiffré.
Sheet / Drive	Oui	companies.sheet_id ; Drive connecté si sheet_owner_mode = client.
Telegram	Non	Code temporaire 15 min puis liaison user/chat unique.

Propriété et révocation

La déconnexion Drive retire l'accès technique et le secret local mais ne supprime jamais le fichier du client. Gmail et Drive conservent des cycles de révocation indépendants.

6. Sélection d’un passage pipeline

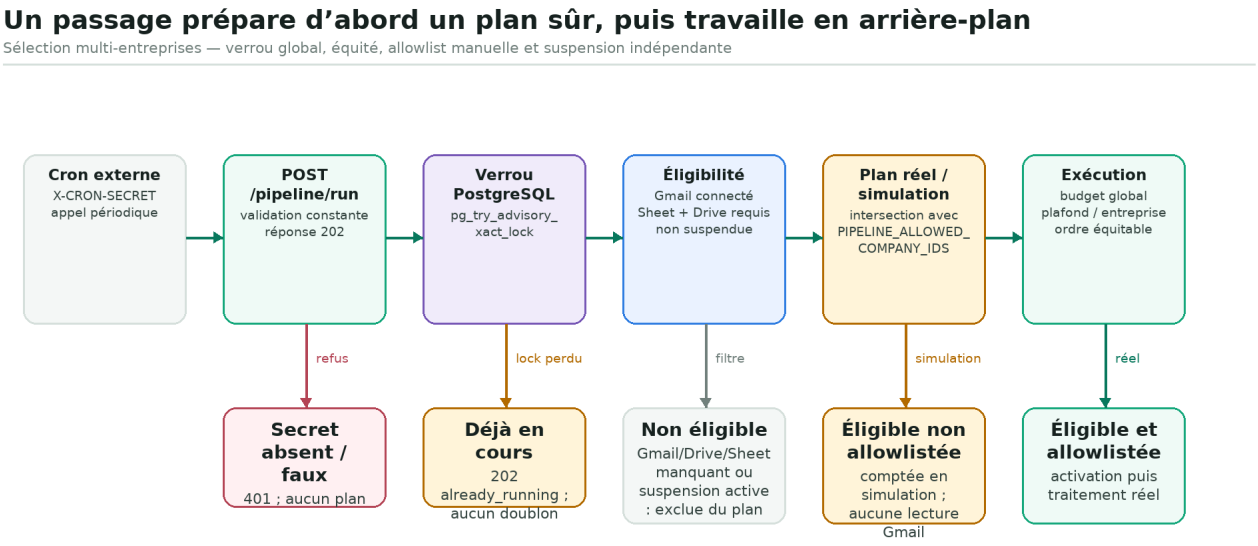


Schéma 6 — Préparation du passage avant la réponse 202, puis exécution sous budget en tâche d’arrière-plan.

Le cron ne lance pas directement une boucle Gmail. Il demande d’abord un plan borné. Un verrou advisory transactionnel empêche deux passages simultanés. Les entreprises sont ordonnées par dernier passage, filtrées par prérequis et suspension, puis intersectées avec PIPELINE_ALLOWED_COMPANY_IDS.

Porte	Si elle échoue	Effet
Secret cron	401.	Aucune lecture ni écriture.
Verrou global	202 already_running.	Le passage en cours reste seul responsable.
Gmail + Sheet/Drive	Entreprise non éligible.	Les autres entreprises continuent.
Suspension active	Entreprise non sélectionnée.	Dashboard et données restent accessibles.
Allowlist manuelle	Plan de simulation uniquement.	Aucun traitement réel pour ce tenant.
Budget temps / plafond	Arrêt propre du passage.	Le prochain cron reprend les éléments non réservés.

7. Traitement d’un email

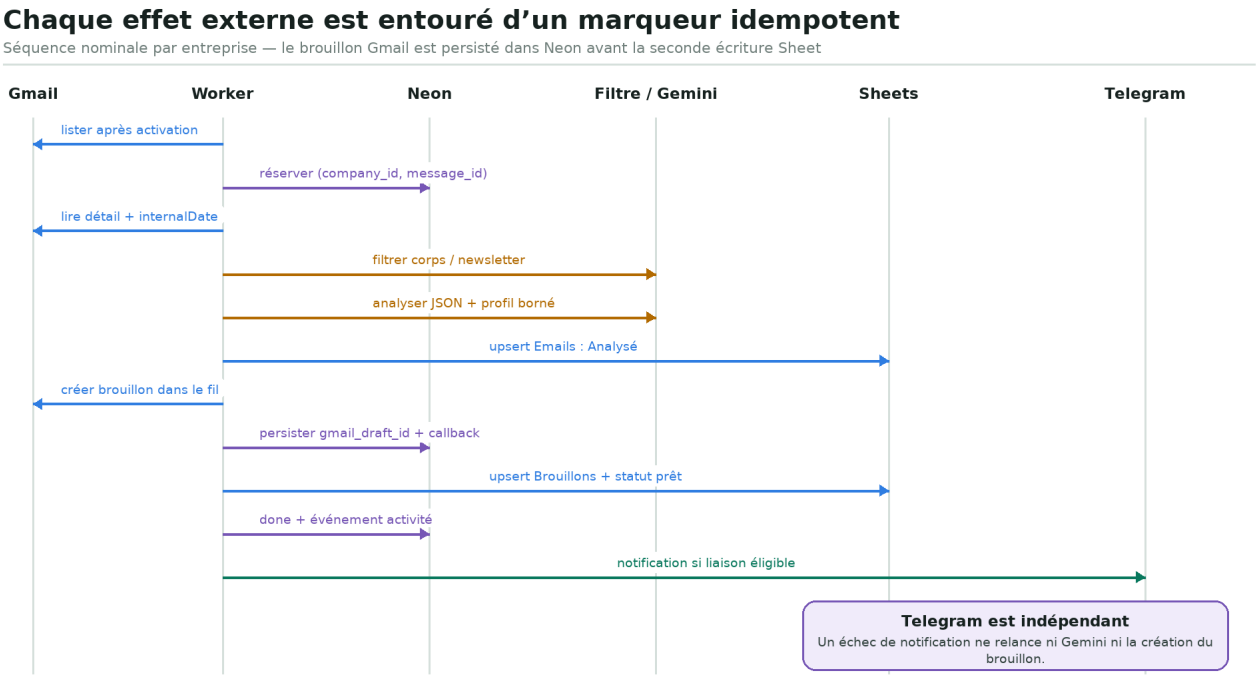


Schéma 7 — Séquence nominale. Le point critique est la persistance du gmail_draft_id avant les écritures suivantes.

- La date pipeline_activated_at est fixée avant le premier accès Gmail ; la requête utilise in:inbox after:<epoch> puis vérifie internalDate.
- La réservation Neon précède la lecture et toute analyse. Le corps est tronqué avant Gemini et le profil entreprise est borné à nouveau à cette frontière.
- La ligne Emails est écrite avant le brouillon. Dès que Gmail renvoie son ID, Neon le conserve dans une transaction distincte ; un retry ne recrée donc pas le brouillon.
- La seconde écriture crée ou met à jour la ligne Brouillons et rattache son ID à Emails. La notification Telegram, si elle est éligible, intervient après l’état done.

Minimisation des effets

Le traitement ne modifie jamais le statut lu/non lu du message Gmail. Aucun contenu d’email ou de brouillon n’est écrit dans les logs du passage.

7.1 Branches de décision et sorties

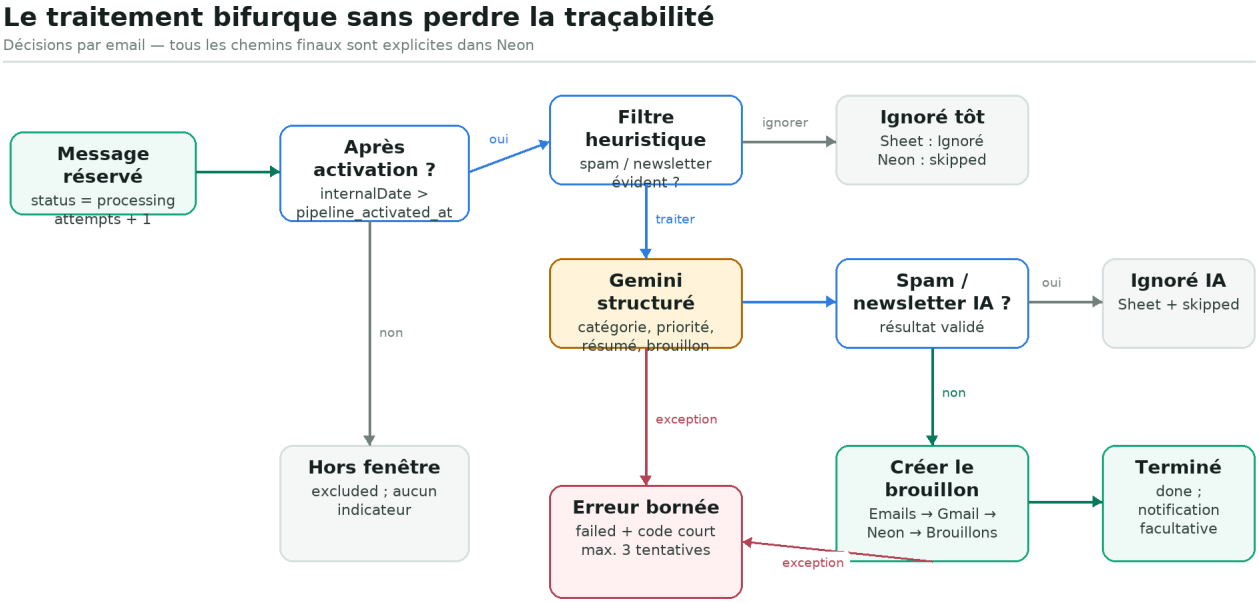


Schéma 8 — Arbre de traitement d’un message réservé : exclusion, filtrage, analyse, brouillon ou échec borné.

Chemin	Neon	Sheet / Gmail
Antérieur à l’activation	excluded	Aucune écriture métier.
Filtré tôt	skipped + événement	Emails = Ignoré ; aucun brouillon.
Spam/newsletter confirmé par Gemini	skipped + catégorie/priorité	Emails = Ignoré ; aucun brouillon.
Pertinent	done + draft_status pending	Emails + brouillon Gmail + Brouillons.
Erreur avant fin	failed + code court	Effets déjà confirmés conservés ; reprise idempotente.
HTTP 429	failed sans coût de tentative	Arrêt de l’entreprise courante pour ce passage.

8. Idempotence et reprises

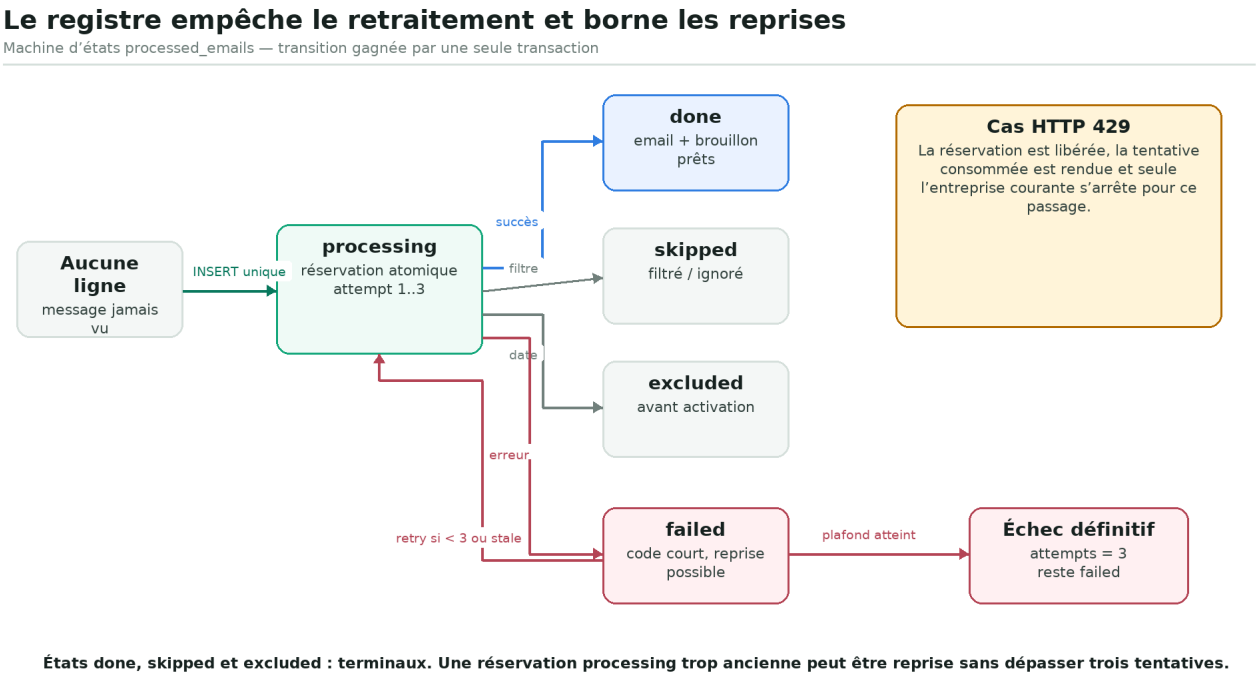


Schéma 9 — Machine d'états du registre email. Les transitions terminales empêchent tout retraitement.

L'unicité tenant/message tranche la concurrence à l'insertion. Un échec ou une réservation processing trop ancienne peut être repris par une mise à jour conditionnelle qui incrémente attempts. Toutes les transitions vérifient record_id, company_id, message_id, statut courant et numéro de tentative.

Risque	Mécanisme actuel
Double cron	Verrou advisory global.
Double message Gmail	Contrainte unique par entreprise.
Crash après création du brouillon	ID Gmail persisté avant la seconde écriture Sheet.
Crash de synchronisation Sheet	Upsert par Email_ID / Draft_ID ; action Gmail non répétée.
Quota 429	Tentative rendue et entreprise arrêtée sans bloquer les autres.
Boucle infinie	Trois tentatives maximum et codes d'erreur bornés.

8.1 Brouillons, notifications et actions Telegram

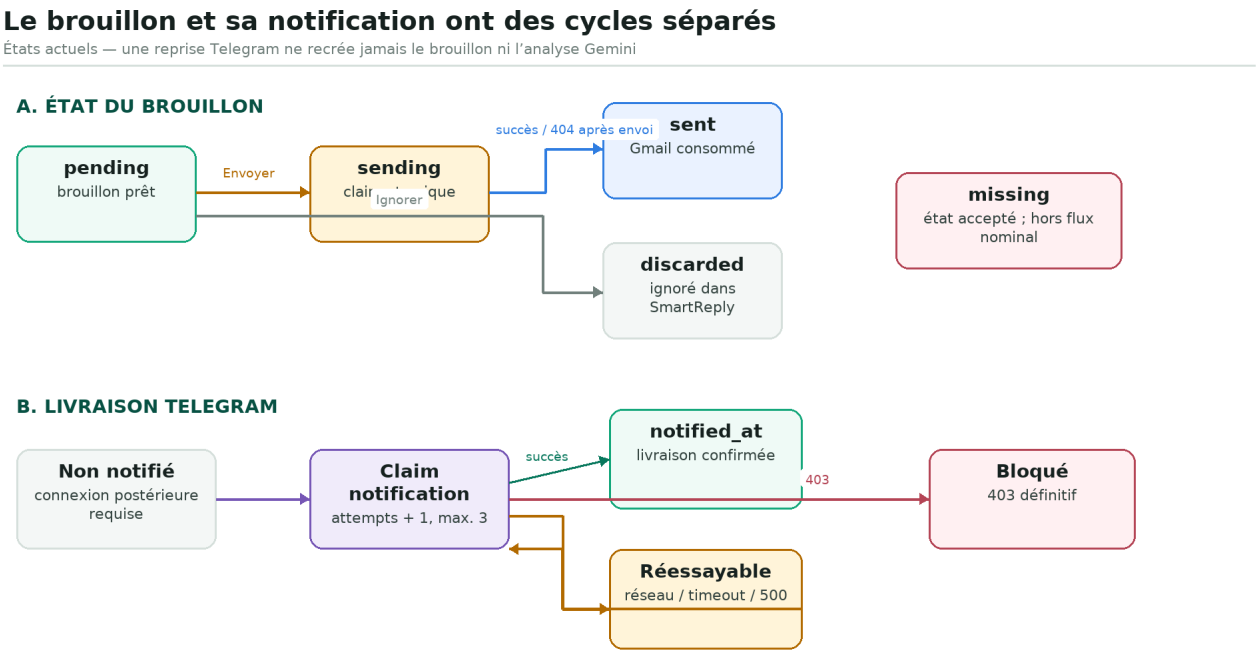
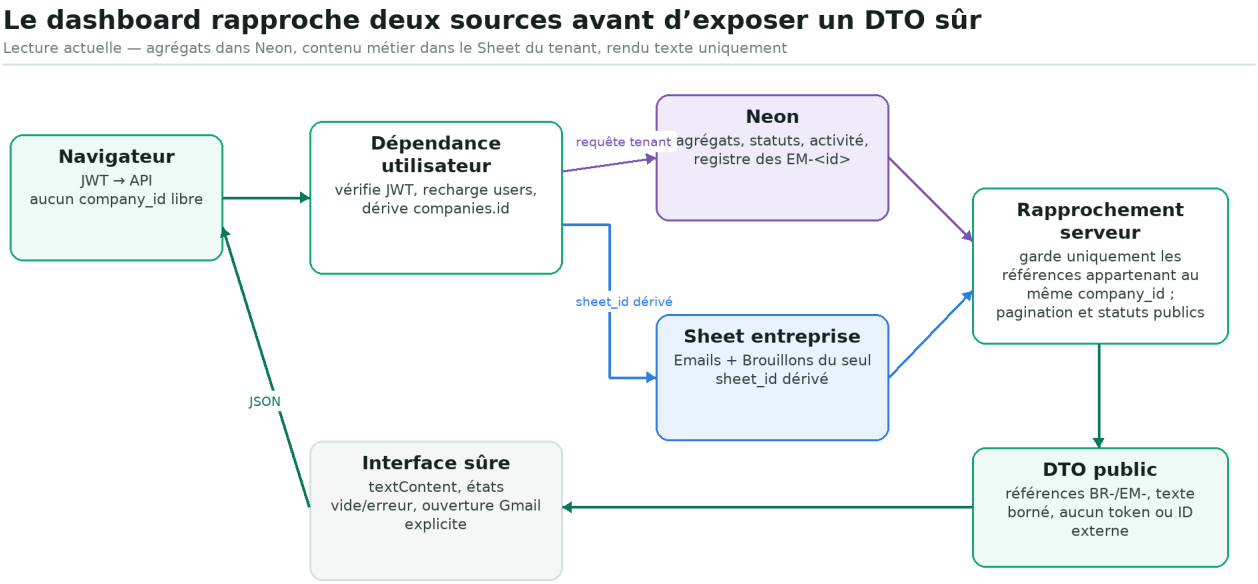


Schéma 10 — Deux machines coordonnées : statut du brouillon et livraison facultative de la notification.

Voir relit le contenu dans le Sheet du tenant. Envoyer prend un claim atomique, utilise le token Gmail de cette entreprise puis synchronise Neon et les deux onglets. Ignorer change le statut connu de SmartReply sans supprimer le brouillon Gmail. Les doubles clics ne répètent pas l'effet Gmail.

Cas ambigu	Traitement
sending trop ancien	Le clic suivant vérifie d'abord l'existence du brouillon dans Gmail.
Gmail répond 404 après drafts.send	Le brouillon est considéré consommé ; état sent sans second envoi.
Telegram timeout / 500	Claim libéré ou repris ; livraison au moins une fois possible dans les cas ambigus.
Telegram 403	notification_blocked_at ; aucune reprise infinie.
Connexion Telegram tardive	Aucun rattrapage des brouillons créés avant connected_at.

9. Modèle de lecture du dashboard



Le dashboard n’interprète pas automatiquement un envoi fait directement dans Gmail ; il affiche les transitions connues de SmartReply.

Schéma 11 — Croisement Neon / Sheet avant exposition. Une panne du Sheet ne masque pas les agrégats Neon.

Vue	Source	Filtrage avant réponse
Accueil / overview	Neon.	company_id dérivé ; agrégats, catégories, priorités et activité.
Emails récents	Sheet Emails + Neon.	Chaque EM-<id> doit exister dans processed_emails du tenant.
Liste Brouillons	Sheet Emails/Brouillons + Neon.	Rapprochement par références internes et pagination serveur.
Détail Brouillon	Même rapprochement.	JWT, tenant, BR-<id>, contenu texte ; aucun ID Gmail ou Sheet.
Ignorer	Neon puis Sheet.	Confirmation UI et claim serveur ; aucune suppression Gmail.

Limite de synchronisation

Les actions effectuées directement dans Gmail ne sont pas interprétées automatiquement comme Envoyé dans le dashboard. Les statuts reflètent seulement les transitions connues de SmartReply.

10. Administration, messages et suspensions

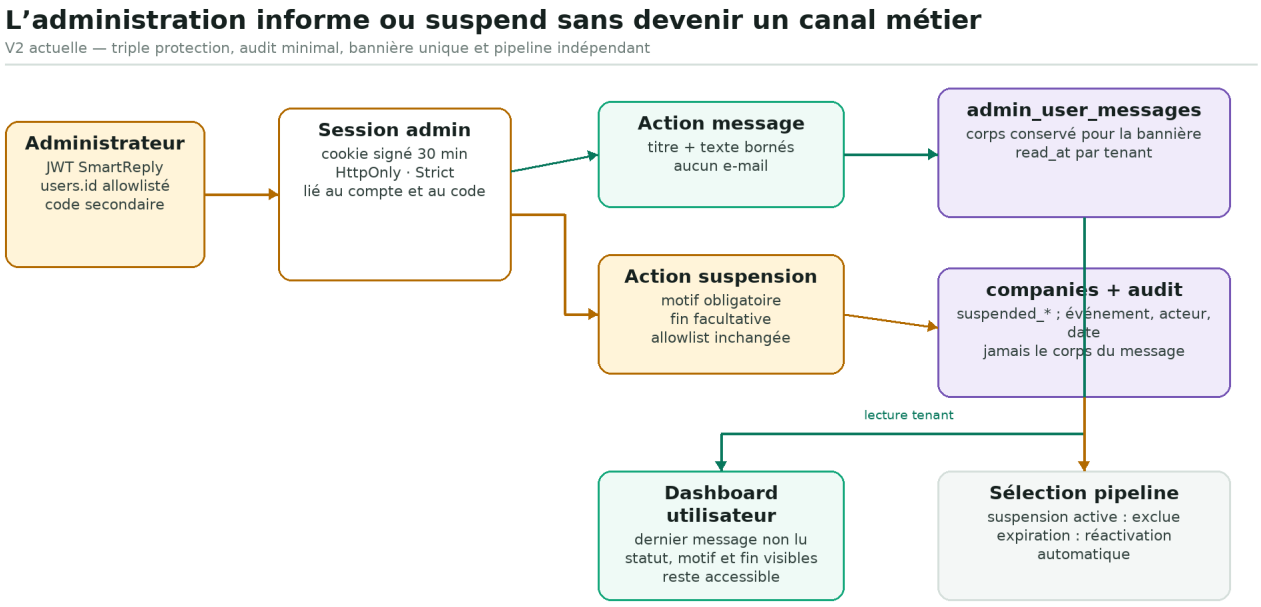


Schéma 12 — Actions V2 bornées. Le message est interne ; la suspension agit seulement sur la sélection du pipeline.

Action	Données écrites	Données visibles à l’utilisateur
Message interne	Titre, corps, auteur, dates ; audit sans corps.	Dernier message non lu sous forme de bannière.
Marquer lu	read_at du message appartenant au tenant.	La bannière disparaît ; aucun historique.
Suspendre	suspended_at, suspended_until, motif ; audit.	Dashboard accessible, motif et fin éventuelle.
Réactiver	Champs de suspension effacés ; audit.	Statut actif ; allowlist inchangée.

Deux axes indépendants

PIPELINE_ALLOWED_COMPANY_IDS reste l’unique validation manuelle. La console ne transforme pas cette liste en registre commercial et ne déclenche ni e-mail, ni relance, ni validation automatique.

11. Sécurité et frontières de confiance

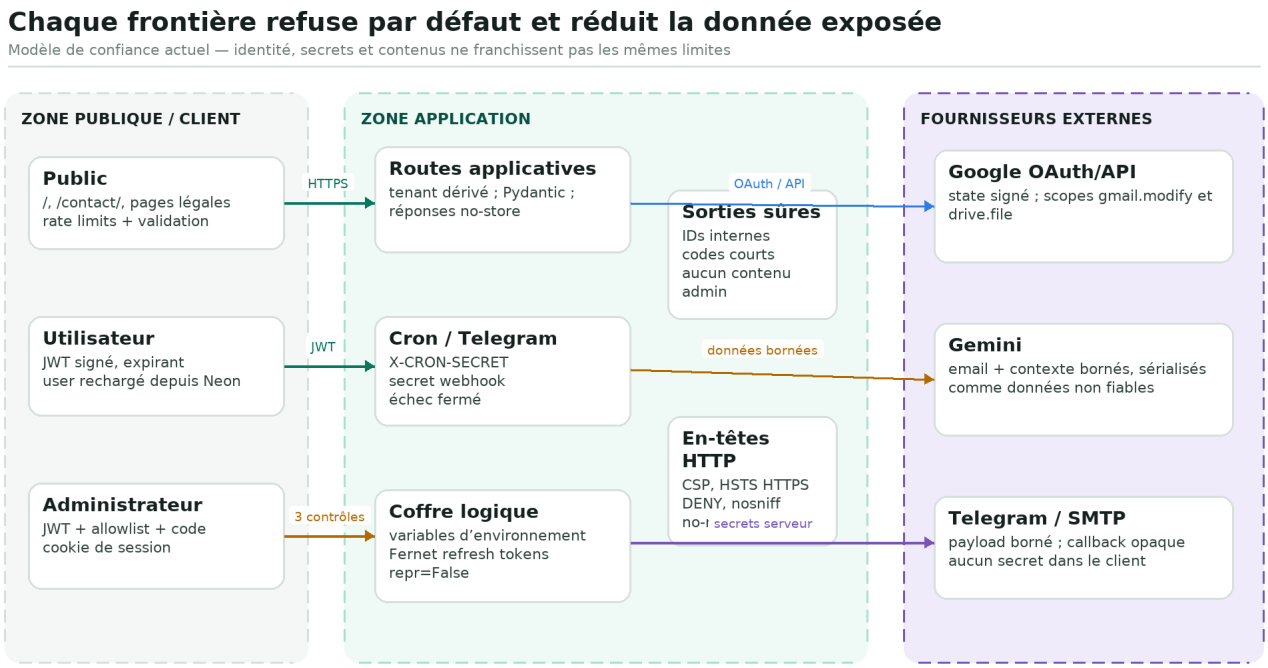


Schéma 13 — Les secrets restent côté serveur et chaque type d'appel possède son propre mécanisme d'authentification.

Surface	Contrôle principal	Réduction d'exposition
Routes utilisateur	JWT HS256 expirant + user rechargé.	Tenant dérivé, réponses no-store, schémas Pydantic.
OAuth Google	state signé + callbacks depuis BASE_URL.	Tokens Fernet, jamais au navigateur.
Pipeline	X-CRON-SECRET + verrou.	Réponse limitée aux IDs internes et compteurs.
Webhook Telegram	Secret Telegram + résolution chat/user/tenant.	Callback opaque sous 64 octets.
Administration	JWT + allowlist + session secondaire.	Aucun secret, ID externe ou contenu métier.
Contact	Validation, rate limit et SMTP SSL.	Champs bornés ; configuration par environnement.

12. Architecture cible post-MVP

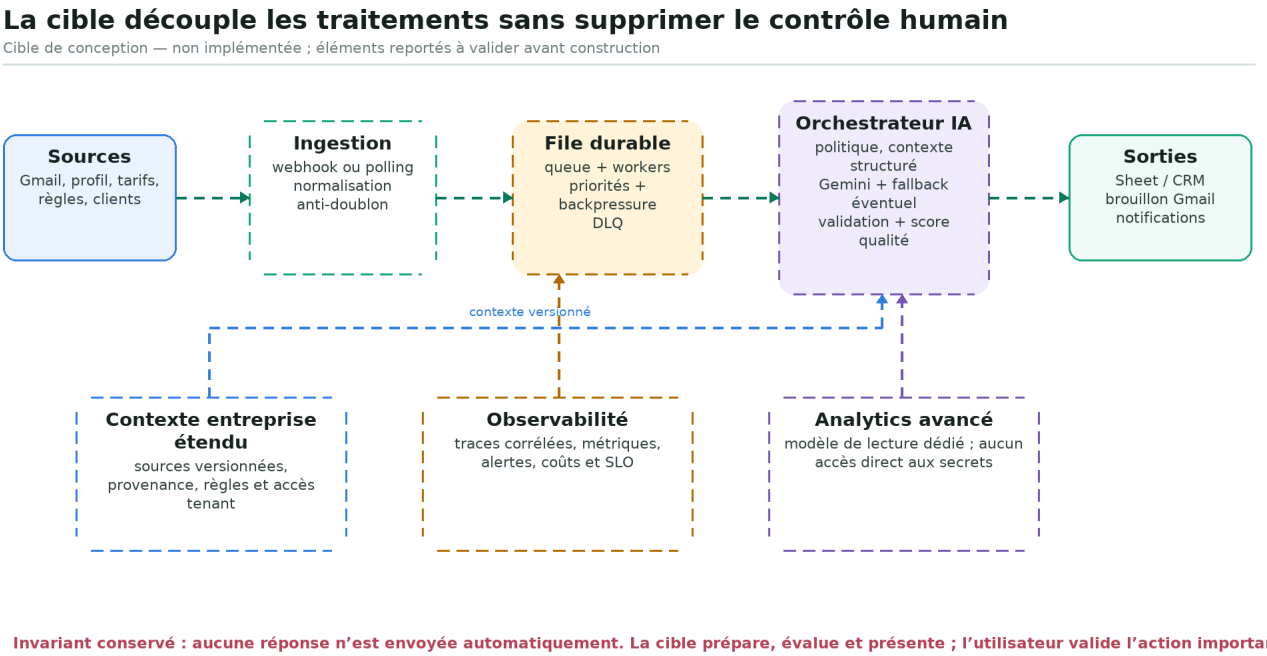


Schéma 14 — Cible conceptuelle. Tous les composants en pointillés sont absents du dépôt actuel et exigent une validation produit/technique.

La direction produit retenue fait évoluer SmartReply d'un assistant de gestion d'emails vers un assistant de communication comprenant davantage le contexte propre à l'entreprise. Le cadrage reporte également une queue de traitement, le fallback multi-modèles, un score qualité et un dashboard avancé. Le schéma organise ces possibilités sans les présenter comme livrées.

Capacité cible	Source	Statut au 6 novembre 2026
Contexte clients / tarifs / règles actif	Fonctions reportées du cadrage.	Non implémenté ; schéma et gouvernance à définir.
Queue durable et workers	Redis/Celery reporté.	Non implémenté ; pipeline actuel en tâche d'arrière-plan FastAPI.
Fallback multi-modèles	Roadmap post-MVP.	Non implémenté ; Gemini est l'unique IA.
Score qualité	Fonction reportée.	Non implémenté ; critères et jeu d'évaluation à définir.
Dashboard avancé	Roadmap post-MVP.	Non implémenté au-delà des vues actuelles.
Observabilité / sauvegarde testée	Besoin d'exploitation identifié.	Procédure et SLO non formalisés dans le dépôt.

Invariant produit

L'envoi automatique sans validation humaine reste explicitement exclu. Une cible plus distribuée ne doit pas déplacer la décision finale vers l'IA ou la queue.

12.1 Trajectoire de construction proposée

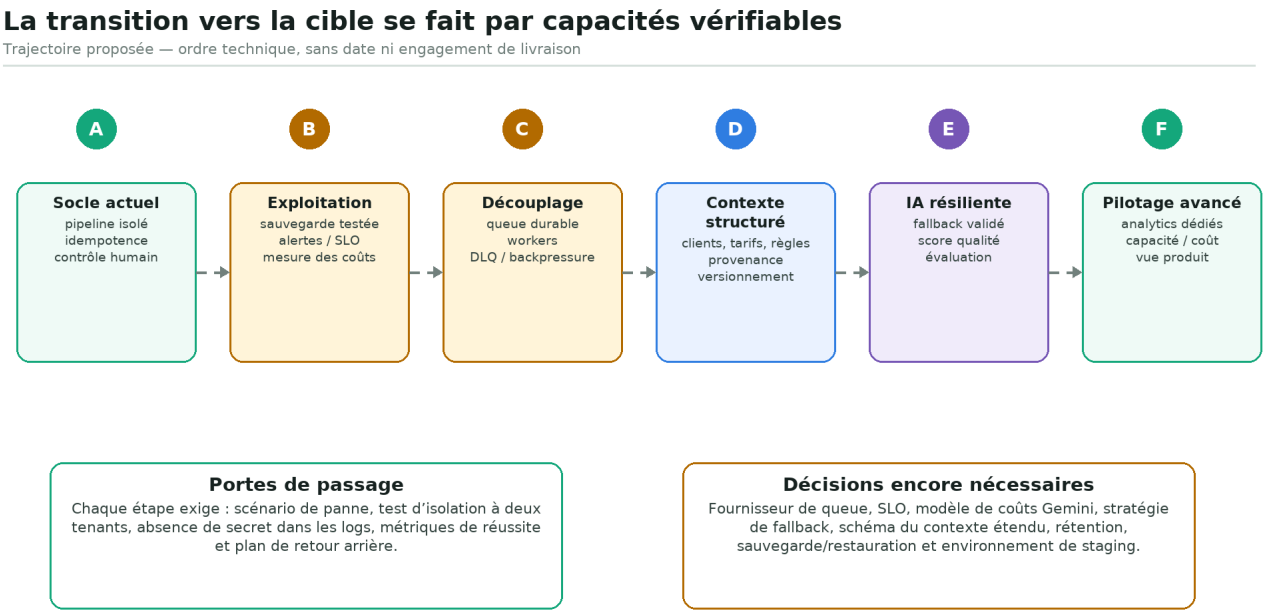


Schéma 15 — Ordre de conception recommandé. Chaque étape doit rester réversible et prouver l'isolation multi-tenant.

La trajectoire commence par l'exploitation du socle existant avant d'introduire de nouveaux composants. Une queue sans observabilité déplacerait seulement les pannes ; un fallback IA sans évaluation pourrait dégrader silencieusement les réponses ; un contexte étendu sans provenance augmenterait le risque de données périmées ou mélangées.

Étape	Critère de sortie minimal
B — Exploitation	Restauration testée, métriques et alerte sur absence de cron / erreurs.
C — Découplage	Sémantique de livraison documentée, DLQ, backpressure et reprise sans doublon.
D — Contexte	Schéma versionné, source/propriétaire, limites de taille, isolation et suppression.
E — IA	Jeu d'évaluation, politique de fallback, budgets de coût et comparaison de qualité.
F — Pilotage	Modèle de lecture dédié, rétention et absence de secret/contenu non nécessaire.

13. Décisions de conception

Décision actuelle	Raison	Alternative écartée ou reportée
Neon pour contrôle, Sheets/Gmail pour métier	Contraintes, transactions et idempotence sans dupliquer le contenu.	Tout stocker dans un Sheet : concurrence et authentification insuffisantes.
Une entreprise dérivée du JWT	Empêche un client de choisir un autre tenant.	company_id libre dans les paramètres.
Un Sheet détenu par le client	Propriété métier claire ; le compte de service n'est que writer.	Fichiers centralisés sur un Drive plateforme pour les nouvelles entreprises.
Brouillon persisté avant la suite	Évite une recréation après panne de Sheet.	Réexécuter tout le pipeline sans checkpoint.
Telegram facultatif	Gmail + Sheet suffisent au cœur de valeur.	Bloquer le traitement tant que 3/3 ne sont pas connectées.
Allowlist manuelle séparée de la suspension	Validation pilote et état opérationnel répondent à deux besoins distincts.	Un unique statut commercial en base.
Interfaces statiques même origine	Pas de build ni de second hébergement pour le MVP.	Framework SSR ou front séparé.
Contrôle humain final	Réduit le risque de communication erronée.	Envoi automatique par l'IA.
Cible distribuée différée	Le volume actuel ne justifie pas encore queue et workers.	Sur-ingénierie avant mesure.

13.1 Règles à préserver lors d'une évolution

- Aucun nouveau composant ne reçoit un company_id non vérifié ; le tenant est propagé depuis une identité authentifiée ou une relation interne.
- Toute écriture externe doit définir son identifiant d'idempotence, son point de checkpoint, son comportement après timeout et son plafond de retry.
- Le contenu d'email, de brouillon ou de profil ne doit pas entrer dans les logs, l'administration ou les métriques sans besoin explicitement validé.
- Une cible asynchrone doit conserver l'ordre logique : réservation → effet externe → preuve persistée → étape suivante.
- Les statuts visibles doivent être dérivés d'événements connus ; SmartReply ne prétend pas connaître une action faite hors de ses canaux.

14. Références et maintenance

Référence	Usage dans ce document
Dépôt smartreply-backend	État actuel au commit 02e779cd57a1f2720aac0b85be3201ebc0d457f5.
README.md	Architecture, pipeline, dashboard, administration, sécurité et roadmap.
Modèles SQLAlchemy + migrations	Entités, contraintes, relations et tête Alembic.
SmartReply_Agent_SaaS_MVP_V1 (2).pdf	Vision, modules, données et fonctions reportées de la cible.
Baraka Caisse — Document de conception.pdf	Référence de structure et de présentation, sans reprise de contenu.
SmartReply Agent — Document technique v1.0	Baseline technique complémentaire.

14.1 Index des schémas

#	Schéma	Objet
1	Architecture actuelle	Composants, stockages et flux.
2	Cas d’utilisation	Acteurs et frontières.
3	Modèle de données	Dix tables et cardinalités.
4	Cartographie des stockages	Classification et minimisation.
5	Onboarding	Compte, Drive, Sheet, Gmail, Telegram.
6	Sélection pipeline	Verrou, éligibilité, allowlist, suspension.
7	Séquence pipeline	Effets externes et checkpoints.
8	Décisions de traitement	Branches par email.
9	États email	Idempotence et reprises.
10	États brouillon / notification	Actions et livraison Telegram.
11	Lecture dashboard	Rapprochement Neon / Sheet.
12	Administration	Messages et suspension.
13	Frontières de sécurité	Authentification et secrets.
14	Architecture cible	Composants post-MVP non implémentés.
15	Trajectoire cible	Ordre de construction proposé.

14.2 Fiche de maintenance

Champ	Valeur
Titre	SmartReply Agent — Document de conception
Version	1.0
Date	6 novembre 2026
Auteur	Koffa Jean Agoudavi
Commit de référence	02e779cd57a1f2720aac0b85be3201ebc0d457f5
Tête Alembic	a9c4e7f2b6d1
Périmètre	État actuel vérifié + cible post-MVP explicitement non implémentée.
Révision	Recalculer tables, routes, statuts et diagrammes après toute migration ou évolution du pipeline.

Source exclue

SMARTREPLY_AGENT_SOURCE_DE_VERITE.md n’est pas utilisé : il est déclaré obsolète. Toute évolution de la cible doit être revalidée à partir du cadrage produit actif et de décisions explicites du Product Owner.